

Back-end em python



Prof^a. Ma. Ana Clara Gomes

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces

Objetivos da Aula

- Entender **encapsulamento** e como proteger atributos de acesso indevido.
- Usar **getters e setters** para controlar acesso a dados.
- Compreender o papel de **classes abstratas** e **interfaces** em Python.
- Criar exemplos práticos de herança e abstração.





Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Introdução Teórica

Encapsulamento

- Ideia: **proteger os dados** de um objeto.
- Em Python, usamos convenções:
 - `atributo` → público
 - `_atributo` → protegido (uso interno)
 - `__atributo` → privado (não acessível diretamente fora da classe).

 Analogia:
Pense numa **conta bancária** → o saldo não pode ser alterado diretamente, apenas por **saque** ou **depósito**.



Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Getters e Setters

- **Getter** → obtém o valor de um atributo.
- **Setter** → altera o valor de forma controlada.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.__nome = nome        # privado
        self.__idade = idade      # privado

    def get_nome(self):
        return self.__nome

    def set_nome(self, novo_nome):
        if len(novo_nome) > 0:
            self.__nome = novo_nome
        else:
            print(" Nome inválido")
```



Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Classes Abstratas

- Uma classe que não pode ser instanciada diretamente.
- Define uma “base” para outras classes herdarem.
- Usa o módulo abc (Abstract Base Class).

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def som(self):
        pass
```



Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces

Interfaces

- Em Python, interfaces são implementadas como classes abstratas apenas com métodos abstratos.
- Forçam as classes filhas a seguirem um contrato.





Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Boas Práticas

- Sempre proteger atributos críticos (`__saldo`).
- Use **propriedades** (`@property`) em vez de getters/setters manuais quando possível:

```
class Pessoa:
    def __init__(self, nome):
        self.__nome = nome

    @property
    def nome(self):
        return self.__nome

    @nome.setter
    def nome(self, novo_nome):
        if novo_nome:
            self.__nome = novo_nome
```




Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces

Aplicações Reais

- **Encapsulamento:** segurança de dados (ex.: saldo de contas, senha de usuários).
- **Classes abstratas:** frameworks (Django, Flask) usam para definir contratos.
- **Interfaces:** sistemas de pagamento, plugins, drivers de dispositivos.



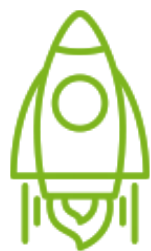


Aula 13 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Exercícios para Sala

1. Crie uma classe **Produto** com atributos privados **nome** e **preco**.
Use **getters** e **setters** para garantir que o preço nunca seja negativo.
2. Crie uma classe abstrata **Funcionario** com método abstrato **calcular_salario()**.
Crie **Gerente** e **Vendedor** que implementem de formas diferentes.
3. Expanda o exemplo da **ContaBancaria**:
Adicione **transferência entre contas**.
Use getters para exibir o saldo.
4. (Desafio) Crie um sistema de transporte:
Classe abstrata **Transporte**.
Classes **Ônibus**, **Metrô** e **Bicicleta** que implementem **viajar()**.



Aula 11 — Encapsulamento, Getters/Setters, Classes Abstratas e Interfaces



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO